Universidad Tecnológica de Pereira
Programación III
Profesor: Ramiro Andrés Barrios Valencia
Integrantes: Julian Guadir, Alejandro Ipaz
Repositorio: https://github.com/julianguadir56-sudo/trabajo-3

Actividad 03 - Casos con listas

1. Introducción

En esta actividad tomamos dos programas que hicimos antes en Prolog, el de la familia Simpson (Actividad 1) y el de las ciudades de Canadá (Actividad 2), y los modificamos para que guarden la información en estructuras y listas en vez de hechos sueltos. Para que el programa siga respondiendo lo mismo tuvimos que cambiar las reglas que leían esos hechos. Los archivos son `simpson_listas.pl` y `canada_listas.pl`, y el código completo está al final en los anexos.

2. Ejercicio 1: familia Simpson

2.1 Cómo estaba antes

Cada relación era un hecho aparte. Había 8 hechos `padre/2`, 8 hechos `madre/2` y un hecho `hombre/1` o `mujer/1` por cada persona:

```
padre(abraham, herbert).
padre(abraham, homero).
padre(clancy, marge).
...
madre(mona, homero).
madre(jacqueline, marge).
...
hombre(abraham).
mujer(mona).
...
```

2.2 Cómo quedó

Ahora cada familia es una estructura `familia(pareja(Padre, Madre), Hijos)` y el último argumento es una lista con los hijos. El género lo pasamos a dos listas, `hombres/1` y `mujeres/1`. Herbert y Ling solo tienen un progenitor en el árbol, así que en el otro lugar de la pareja pusimos `desconocido`.

```
familia(pareja(abraham,     desconocido), [herbert]).
familia(pareja(abraham,     mona),        [homero]).
familia(pareja(clancy,      jacqueline),  [marge, patty, selma]).
familia(pareja(homero,      marge),       [bart, lisa, maggie]).
familia(pareja(desconocido, selma),       [ling]).

hombres([abraham, herbert, homero, clancy, bart]).
mujeres([mona, jacqueline, marge, patty, selma, lisa, maggie, ling]).
```

2.3 Reglas que cambiamos

Solo cambiaron cuatro predicados: `hombre/1`, `mujer/1`, `padre/2` y `madre/2`. Antes eran hechos y ahora son reglas que buscan en las listas con `member/2`:

```
hombre(X) :- hombres(L), member(X, L).
mujer(X)  :- mujeres(L), member(X, L).

padre(P, H) :- familia(pareja(P, _), Hijos), P \== desconocido, member(H, Hijos).
madre(M, H) :- familia(pareja(_, M), Hijos), M \== desconocido, member(H, Hijos).
```

Con eso no tuvimos que tocar el resto de reglas (`progenitor`, `abuelo`, `tio`, `primo`, `ancestro`, etc.), porque todas se apoyan en `progenitor/2` y este sigue recibiendo los mismos pares de siempre. Además escribimos las familias en el mismo orden de los hechos originales para que las respuestas salgan en el mismo orden.

2.4 Reglas nuevas con listas

Aprovechando las listas agregamos algunas reglas que devuelven todos los resultados de una vez. Usamos `setof/3` para que la lista salga ordenada y sin repetidos, y si no hay resultados se devuelve `[]` en vez de fallar.

```
hijos(P, L)            :- setof(H, progenitor(P, H), L), !.
hijos(_, []).
hermanos_de(X, L)      :- setof(Y, hermanos(X, Y), L), !.
hermanos_de(_, []).
abuelos_de(N, L)       :- setof(A, P^(progenitor(A, P), progenitor(P, N)), L), !.
abuelos_de(_, []).
ancestros_de(D, L)     :- setof(A, ancestro(A, D), L), !.
ancestros_de(_, []).
descendientes_de(A, L) :- setof(D, ancestro(A, D), L), !.
descendientes_de(_, []).
cantidad_hijos(P, N)   :- hijos(P, L), length(L, N).
```

2.5 Pruebas

Las primeras 14 consultas son las de la Actividad 1 y las últimas 6 son de las reglas nuevas. Es lo que muestra SWI-Prolog al ir pulsando `;`.

```
?- padre(homero, bart).
true.

?- abuelo(X, bart).
X = abraham ;
X = clancy.

?- abuela(X, lisa).
X = mona ;
X = jacqueline.

?- hermano(bart, lisa).
true.

?- hermana(X, bart).
X = lisa ;
X = maggie.

?- tio(herbert, bart).
true.
```

```
?- tia(X, bart).
X = patty ;
X = selma.

?- tia(marge, ling).
true.

?- primo(bart, ling).
true.

?- prima(X, ling).
X = lisa ;
X = maggie.

?- sobrino(X, patty).
X = bart.

?- ancestro(abraham, X).
X = herbert ;
X = homero ;
X = bart ;
X = lisa ;
X = maggie.

?- descendiente(ling, X).
X = selma ;
X = clancy ;
X = jacqueline.

?- abuelo(clancy, ling).
true.

?- hijos(homero, L).
L = [bart,lisa,maggie].

?- hermanos_de(bart, L).
L = [lisa,maggie].

?- abuelos_de(bart, L).
L = [abraham,clancy,jacqueline,mona].

?- ancestros_de(ling, L).
L = [clancy,jacqueline,selma].

?- descendientes_de(abraham, L).
L = [bart,herbert,homero,lisa,maggie].

?- cantidad_hijos(clancy, N).
N = 3.
```

3. Ejercicio 2: ciudades de Canadá

3.1 Cómo estaba antes

El grafo eran 9 hechos `arista(Origen, Destino, Costo)` y 6 hechos `nodo(Ciudad)`:

```
arista(vancouver, edmonton, 16).
arista(vancouver, calgary, 13).
arista(calgary, edmonton, 4).
...
nodo(vancouver).
nodo(edmonton).
...
```

3.2 Cómo quedó

Ahora todo el grafo es una sola lista de estructuras `arista/3` y las ciudades son otra lista:

```
grafo([ arista(vancouver, edmonton, 16),  arista(vancouver, calgary,  13),
        arista(calgary,   edmonton,  4),  arista(edmonton,  saskatoon, 12),
        arista(saskatoon, calgary,   9),  arista(calgary,   regina,   14),
        arista(regina,    saskatoon, 7),  arista(saskatoon, winnipeg, 20),
        arista(regina,    winnipeg,  4) ]).

nodos([vancouver, edmonton, calgary, saskatoon, regina, winnipeg]).
```

3.3 Reglas que cambiamos

Igual que en el ejercicio anterior, solo `arista/3` y `nodo/1` pasaron de hechos a reglas que buscan en la lista:

```
arista(O, D, C) :- grafo(G), member(arista(O, D, C), G).
nodo(N)         :- nodos(L), member(N, L).
```

Las reglas `camino/3`, `conectado/3`, `tiene_aristas/1` y `costo_via/4` las dejamos como estaban, porque llaman a `arista/3` igual que antes. El control de ciclos de `camino/3` ya usaba una lista de visitados.

3.4 Reglas nuevas con listas

`ruta/4` hace lo mismo que `camino/3` pero además devuelve la lista de ciudades por las que pasa. Las demás devuelven listas de vecinos, de ciudades alcanzables, de costos y el camino de menor costo.

```
ruta(X, Y, R, C) :- ruta(X, Y, [X], R, C).
ruta(X, Y, V, R, C) :- arista(X, Y, C), reverse([Y|V], R).
ruta(X, Y, V, R, C) :- arista(X, Z, C1), Z \== Y, \+ member(Z, V),
                       ruta(Z, Y, [Z|V], R, C2), C is C1 + C2.

vecinos(X, L)     :- findall(Y, arista(X, Y, _), L).
alcanzables(X, L) :- setof(Y, C^conectado(X, Y, C), L), !.
alcanzables(_, []).
costos(X, Y, L)   :- setof(C, camino(X, Y, C), L), !.
costos(_, _, []).
camino_minimo(X, Y, R, C) :- findall(C0-R0, ruta(X, Y, R0, C0), Pares),
                             Pares \== [], keysort(Pares, [C-R|_]).
```

3.5 Pruebas

Las primeras 5 consultas son las de la Actividad 2 y las últimas 5 son de las reglas nuevas. Notamos que `costo_via(vancouver, calgary, winnipeg, C)` da varios costos porque combina todos los caminos de Vancouver a Calgary con todos los de Calgary a Winnipeg. El programa original hacía lo mismo.

```
?- camino(saskatoon, vancouver, C).
false.

?- conectado(regina, N, C).
N = edmonton, C = 20 ;
N = calgary, C = 16 ;
N = saskatoon, C = 7 ;
N = winnipeg, C = 4 ;
```

```
N = winnipeg, C = 27.

?- tiene_aristas(regina).
true.

?- costo_via(vancouver, calgary, winnipeg, C).
C = 49 ;
C = 31 ;
C = 54 ;
C = 73 ;
C = 55 ;
C = 78.

?- camino(edmonton, calgary, C).
C = 21.

?- ruta(edmonton, calgary, R, C).
R = [edmonton,saskatoon,calgary], C = 21.

?- vecinos(calgary, L).
L = [edmonton,regina].

?- alcanzables(regina, L).
L = [calgary,edmonton,saskatoon,winnipeg].

?- costos(vancouver, winnipeg, L).
L = [31,48,49,54,55].

?- camino_minimo(vancouver, winnipeg, R, C).
R = [vancouver,calgary,regina,winnipeg], C = 31.
```

4. Comparación con los programas originales

Para estar seguros de que el cambio no dañó nada, cargamos en SWI-Prolog el programa original y el nuevo y corrimos las mismas consultas en los dos, sacando todas las soluciones de cada una. Probamos 28 consultas en Simpson y 15 en Canadá (las de las actividades más varias consultas abiertas, con variables en todos los lugares) y en todas salieron las mismas respuestas y en el mismo orden.

Un detalle: algunas consultas, como `hermana(X, bart)`, devuelven a la misma persona dos veces porque Bart y Lisa comparten padre y madre, y la regla `hermanos` los encuentra por cada progenitor. Ya pasaba en la Actividad 1 y lo dejamos así para no cambiar el comportamiento. En las pruebas de arriba quitamos los repetidos para que se lea mejor. Si se quiere la lista limpia, `hermanos_de/2` ya la da sin repetidos.

5. Conclusiones

Guardar los datos en estructuras dentro de listas hace que la información quede junta (una lista de aristas, una lista de hijos por familia) y no repartida en muchos hechos. Lo que más nos ayudó fue dejar los predicados con el mismo nombre y la misma cantidad de argumentos: como las demás reglas solo los usan y no saben cómo están guardados los datos, casi todo el programa siguió funcionando sin cambios. También vimos que con listas se pueden devolver resultados completos, como todos los hijos de alguien o la ruta recorrida entre dos ciudades, usando `findall/3` y `setof/3`.

Anexo A. Código de simpson_listas.pl

```
% ======================================================================
% Universidad Tecnologica de Pereira - Programacion III
% Actividad03: Casos con Listas
% EJERCICIO 1: ARBOL GENEALOGICO DE LOS SIMPSON (con estructuras y listas)
% Integrantes: Julian Guadir, Alejandro Ipaz
% ======================================================================
%
% CAMBIO RESPECTO A LA ACTIVIDAD 1:
%   Antes: un hecho suelto por cada relacion (padre/2, madre/2).
%   Ahora: los datos viven en ESTRUCTURAS y LISTAS:
%     - familia(pareja(Padre, Madre), Hijos)   -> estructura con una lista de hijos
%     - hombres([...]) y mujeres([...])        -> listas con el genero
%   padre/2 y madre/2 pasan de ser hechos a ser REGLAS que recorren esas listas.
%   Todas las demas reglas (abuelo, tio, primo, ancestro...) se mantienen
%   igual, por eso el programa responde las mismas consultas.
% ======================================================================

% ---------- Base de conocimiento: estructuras y listas ----------

% 'desconocido' marca el progenitor que no aparece en el arbol.
familia(pareja(abraham,     desconocido), [herbert]).
familia(pareja(abraham,     mona),        [homero]).
familia(pareja(clancy,      jacqueline),  [marge, patty, selma]).
familia(pareja(homero,      marge),       [bart, lisa, maggie]).
familia(pareja(desconocido, selma),       [ling]).

hombres([abraham, herbert, homero, clancy, bart]).
mujeres([mona, jacqueline, marge, patty, selma, lisa, maggie, ling]).

% ---------- Reglas sobre las listas ----------

hombre(X) :- hombres(L), member(X, L).
mujer(X)  :- mujeres(L), member(X, L).

% padre(P, H): P es el padre de H (primer elemento de la pareja)
padre(P, H) :-
    familia(pareja(P, _), Hijos),
    P \== desconocido,
    member(H, Hijos).

% madre(M, H): M es la madre de H (segundo elemento de la pareja)
madre(M, H) :-
    familia(pareja(_, M), Hijos),
    M \== desconocido,
    member(H, Hijos).

% ---------- Reglas de la Actividad 1 (sin cambios) ----------

% progenitor(P, H): P es padre o madre de H
progenitor(P, H) :- padre(P, H).
progenitor(P, H) :- madre(P, H).

hijo(H, P) :- hombre(H), progenitor(P, H).
hija(H, P) :- mujer(H), progenitor(P, H).

% abuelos (2 generaciones)
abuelo(A, N) :- hombre(A), progenitor(A, P), progenitor(P, N).
abuela(A, N) :- mujer(A), progenitor(A, P), progenitor(P, N).

% hermanos (comparten al menos un progenitor)
hermanos(X, Y) :- progenitor(P, X), progenitor(P, Y), X \== Y.
```

```
hermano(X, Y) :- hombre(X), hermanos(X, Y).
hermana(X, Y) :- mujer(X), hermanos(X, Y).

% tios: hermanos de un progenitor
tio(T, S) :- hombre(T), progenitor(P, S), hermanos(T, P).
tia(T, S) :- mujer(T), progenitor(P, S), hermanos(T, P).

% sobrinos
sobrino(S, T) :- hombre(S), progenitor(P, S), hermanos(T, P).
sobrina(S, T) :- mujer(S), progenitor(P, S), hermanos(T, P).

% primos: hijos de hermanos
primos(X, Y) :- progenitor(P, X), progenitor(Q, Y), hermanos(P, Q).
primo(X, Y) :- hombre(X), primos(X, Y).
prima(X, Y) :- mujer(X), primos(X, Y).

% ancestros y descendientes (recursivas)
ancestro(A, D) :- progenitor(A, D).
ancestro(A, D) :- progenitor(A, X), ancestro(X, D).
descendiente(D, A) :- ancestro(A, D).

% ---------- Reglas nuevas que devuelven LISTAS ----------

% hijos(P, L): lista de los hijos de P (sin repetidos)
hijos(P, L) :- setof(H, progenitor(P, H), L), !.
hijos(_, []).

% hermanos_de(X, L): lista de hermanos de X
hermanos_de(X, L) :- setof(Y, hermanos(X, Y), L), !.
hermanos_de(_, []).

% abuelos_de(N, L): lista de abuelos y abuelas de N
abuelos_de(N, L) :-
    setof(A, P^(progenitor(A, P), progenitor(P, N)), L), !.
abuelos_de(_, []).

% ancestros_de(D, L): lista de todos los ancestros de D
ancestros_de(D, L) :- setof(A, ancestro(A, D), L), !.
ancestros_de(_, []).

% descendientes_de(A, L): lista de todos los descendientes de A
descendientes_de(A, L) :- setof(D, ancestro(A, D), L), !.
descendientes_de(_, []).

% cantidad_hijos(P, N): numero de hijos de P
cantidad_hijos(P, N) :- hijos(P, L), length(L, N).

% ---------- Consultas de prueba (resultado esperado) ----------
% Las mismas de la Actividad 1:
% ?- padre(homero, bart).        % true
% ?- abuelo(X, bart).            % X = abraham ; X = clancy
% ?- abuela(X, lisa).            % X = mona ; X = jacqueline
% ?- hermano(bart, lisa).        % true
% ?- hermana(X, bart).           % X = lisa ; X = maggie
% ?- tio(herbert, bart).         % true
% ?- tia(X, bart).               % X = patty ; X = selma
% ?- tia(marge, ling).           % true
% ?- primo(bart, ling).          % true
% ?- prima(X, ling).             % X = lisa ; X = maggie
% ?- sobrino(X, patty).          % X = bart
% ?- ancestro(abraham, X).       % herbert, homero, bart, lisa, maggie
% ?- descendiente(ling, X).      % X = selma ; X = clancy ; X = jacqueline
% ?- abuelo(clancy, ling).       % true
% Nuevas (con listas):
```

```
% ?- hijos(homero, L).           % L = [bart, lisa, maggie]
% ?- hermanos_de(bart, L).       % L = [lisa, maggie]
% ?- abuelos_de(bart, L).        % L = [abraham, clancy, jacqueline, mona]
% ?- ancestros_de(ling, L).      % L = [clancy, jacqueline, selma]
% ?- descendientes_de(abraham, L). % L = [bart, herbert, homero, lisa, maggie]
% ?- cantidad_hijos(clancy, N).  % N = 3
```

Anexo B. Código de canada_listas.pl

```
% ======================================================================
% Universidad Tecnologica de Pereira - Programacion III
% Actividad03: Casos con Listas
% EJERCICIO 2: CIUDADES DE CANADA - GRAFO DIRIGIDO (con estructuras y listas)
% Integrantes: Julian Guadir, Alejandro Ipaz
% ======================================================================
%
% CAMBIO RESPECTO A LA ACTIVIDAD 2:
%   Antes: un hecho arista/3 por cada conexion y un hecho nodo/1 por ciudad.
%   Ahora: el grafo completo es UNA lista de estructuras arista(O, D, Costo)
%   y los nodos son UNA lista de ciudades. arista/3 y nodo/1 pasan a ser
%   REGLAS que recorren esas listas, asi camino/3, conectado/3,
%   tiene_aristas/1 y costo_via/4 funcionan sin cambios en su uso.
% ======================================================================

% ---------- Base de conocimiento: estructuras y listas ----------

% grafo(Aristas): lista de estructuras arista(Origen, Destino, Costo)
grafo([ arista(vancouver, edmonton, 16),
        arista(vancouver, calgary,  13),
        arista(calgary,   edmonton,  4),
        arista(edmonton,  saskatoon, 12),
        arista(saskatoon, calgary,   9),
        arista(calgary,   regina,   14),
        arista(regina,    saskatoon, 7),
        arista(saskatoon, winnipeg, 20),
        arista(regina,    winnipeg,  4) ]).

% nodos(Ciudades): lista de vertices del grafo
nodos([vancouver, edmonton, calgary, saskatoon, regina, winnipeg]).

% ---------- Reglas sobre las listas ----------

% arista(O, D, C): hay una conexion de O a D con costo C
arista(O, D, C) :- grafo(G), member(arista(O, D, C), G).

% nodo(N): N es una ciudad del grafo
nodo(N) :- nodos(L), member(N, L).

% ---------- Reglas de la Actividad 2 (misma logica) ----------

% camino(X, Y, C): existe un camino de X a Y con costo total C.
% Se lleva una lista de visitados para evitar ciclos.
camino(X, Y, C) :- camino(X, Y, [X], C).

camino(X, Y, _, C) :-
    arista(X, Y, C).
camino(X, Y, V, C) :-
    arista(X, Z, C1),
    Z \== Y,
    \+ member(Z, V),
    camino(Z, Y, [Z|V], C2),
    C is C1 + C2.

% conectado(X, Y, C): nodos alcanzables desde X y el costo de cada ruta.
conectado(X, Y, C) :-
    nodo(Y),
    X \== Y,
    camino(X, Y, C).

% tiene_aristas(X): el nodo X tiene al menos una arista (saliente o entrante).
```

```
tiene_aristas(X) :- arista(X, _, _), !.
tiene_aristas(X) :- arista(_, X, _), !.

% costo_via(X, Y, Z, C): costo de ir de X a Z pasando por Y.
costo_via(X, Y, Z, C) :-
    camino(X, Y, C1),
    camino(Y, Z, C2),
    C is C1 + C2.

% ---------- Reglas nuevas que usan LISTAS ----------

% ruta(X, Y, R, C): igual que camino/3 pero R es la LISTA de ciudades
% recorridas, de X a Y.
ruta(X, Y, R, C) :- ruta(X, Y, [X], R, C).

ruta(X, Y, V, R, C) :-
    arista(X, Y, C),
    reverse([Y|V], R).
ruta(X, Y, V, R, C) :-
    arista(X, Z, C1),
    Z \== Y,
    \+ member(Z, V),
    ruta(Z, Y, [Z|V], R, C2),
    C is C1 + C2.

% vecinos(X, L): lista de ciudades a las que se llega directo desde X
vecinos(X, L) :- findall(Y, arista(X, Y, _), L).

% alcanzables(X, L): lista (sin repetidos) de ciudades alcanzables desde X
alcanzables(X, L) :- setof(Y, C^conectado(X, Y, C), L), !.
alcanzables(_, []).

% costos(X, Y, L): lista ordenada de los costos de todos los caminos X -> Y
costos(X, Y, L) :- setof(C, camino(X, Y, C), L), !.
costos(_, _, []).

% camino_minimo(X, Y, R, C): ruta de menor costo entre X e Y
camino_minimo(X, Y, R, C) :-
    findall(C0-R0, ruta(X, Y, R0, C0), Pares),
    Pares \== [],
    keysort(Pares, [C-R|_]).

% ---------- Consultas de prueba (resultado esperado) ----------
% Las mismas de la Actividad 2:
% ?- camino(saskatoon, vancouver, C).        % false
% ?- conectado(regina, N, C).                % N = saskatoon, C = 7 ; ...
% ?- tiene_aristas(regina).                  % true
% ?- costo_via(vancouver, calgary, winnipeg, C).
% ?- camino(edmonton, calgary, C).           % C = 21
% Nuevas (con listas):
% ?- ruta(edmonton, calgary, R, C).          % R = [edmonton, saskatoon, calgary], C = 21
% ?- vecinos(calgary, L).                    % L = [edmonton, regina]
% ?- alcanzables(regina, L).
% ?- camino_minimo(vancouver, winnipeg, R, C).
```